

AI CODING BOILERPLATE

One source of truth for every coding agent.

A drop-in project skeleton that makes Claude Code, Cursor, GitHub Copilot, Windsurf, Cline, Aider, and any AGENTS.md-aware agent productive on your repo — from a single set of templates you maintain.

Contents

1. The problem this solves
2. The solution at a glance
3. Who it's for & when to use it
4. Benefits
5. Project layout
6. How it works under the hood
7. Quick start (5 minutes)
8. Filling in the templates
9. Working with skills
10. Architecture decision records
11. Token optimization in depth
12. Adding a new agent
13. Best practices
14. FAQ & troubleshooting
15. File reference

1. The problem this solves

Teams that use AI coding assistants quickly hit the same wall. Every assistant wants its own configuration file in its own location, with its own format:

- **Claude Code** reads `CLAUDE.md`.
- **Cursor** reads `.cursorrules` (legacy) or `.cursor/rules/*.mdc` (modern).
- **GitHub Copilot** reads `.github/copilot-instructions.md`.
- **Windsurf** reads `.windsurfrules`.
- **Cline** reads `.clinerules`.
- **Aider** reads `CONVENTIONS.md`.
- **Codex, Jules, OpenAI Agents, and others** are converging on `AGENTS.md`.

Each file ends up saying roughly the same thing — "our stack is X, our test command is Y, never commit secrets, follow our naming conventions" — and each one drifts out of sync as the project evolves. A change to the test command lives in one file; a new architectural rule gets added to another; a junior engineer copies the wrong one as a template.

Worse, the files that *do* get maintained tend to grow into 1,500-line monoliths because nobody knows what's safe to remove. That long file then gets injected into the assistant's context on *every single turn*, burning tokens and dulling the model's focus on what the user actually asked.

The fundamental issue: most teams treat AI configuration like environment variables — one per tool, copied around, drifting. It should be treated like a shared library: one source of truth, many thin consumers.

2. The solution at a glance

This boilerplate inverts the usual layout. Instead of N agent configs each containing the full picture, you maintain one `.ai/` folder of templates, and every agent's config file is a thin shim that delegates into it.

<code>.ai/</code>	← you edit this (source of truth)
├─ <code>project.md</code>	what the project is, stack, commands
├─ <code>rules.md</code>	hard do / don't list
├─ <code>architecture.md</code>	system design (loaded on demand)
├─ <code>style.md</code>	naming / formatting (loaded on demand)
├─ <code>glossary.md</code>	domain terms (loaded on demand)
├─ <code>skills/</code>	step-by-step playbooks
└─ <code>decisions/</code>	architecture decision records
 <code>AGENTS.md</code>	 universal entry-point (small, always loaded)
 <code>CLAUDE.md</code>	
<code>.cursorrules</code>	┌ thin shims that ├ point into <code>.ai/</code> └ (you rarely touch these)
<code>.cursor/rules/project.mdc</code>	
<code>.github/copilot-instructions.md</code>	
<code>.windsurfrules</code>	
<code>.clinerules</code>	
<code>CONVENTIONS.md (+ .aider.conf.yml)</code>	

You edit files in `.ai/` (and `AGENTS.md`). The agent-specific shims are already wired up: each one tells its respective tool to read `AGENTS.md` plus the relevant files inside `.ai/`.

Add a rule to `.ai/rules.md` and it's instantly visible to every agent. Update the test command in `.ai/project.md` and every assistant runs the right thing. No drift, no copy-paste.

3. Who it's for & when to use it

Primary use cases

- **Solo developers who switch between assistants.** You use Claude Code for deep work, Cursor for inline editing, and Copilot in your CI. One set of templates, all three tools aligned.
- **Teams where engineers prefer different assistants.** Alex uses Cursor, Sam uses Claude Code, Riley uses Windsurf. Everyone gets the same guardrails without arguing about whose config is canonical.
- **Open-source maintainers.** Contributors arrive with whatever AI tool they prefer. With this layout, you write the rules once and any contributor's assistant picks them up.
- **"Vibe coding" projects.** Side projects, prototypes, weekend builds — where you want the AI to make sensible choices without constant correction, but you don't want to write 5 config files for a 5-file project.
- **Codebases that change tools every 6 months.** The AI tooling landscape is moving fast. When the next assistant arrives, add one shim file and you're done — no template rewrite.

When it's overkill

- A single-file script or a throwaway experiment. Just write a quick comment at the top.
- A codebase where no one uses AI assistants. (Then why are you reading this?)
- A codebase with a strict single-tool mandate that will never change. You can still benefit, but the multi-agent layer is unused weight.

What this boilerplate is *not*

- Not a code framework. It contains no application code, no `package.json`, no `requirements.txt`. Drop it into any repo regardless of language.
- Not a prompt collection. It's a place for *your* rules, not generic prompts harvested from the internet.
- Not opinionated about your stack. Templates are language-agnostic with placeholders for you to fill.

4. Benefits

1. No drift between agent configs

Change the test command once in `.ai/project.md` and every agent runs the right thing. There's no "the Cursor config says `npm test` but we moved to `pnpm test` six months ago" failure mode.

2. Token efficiency by design

Always-loaded files (`AGENTS.md` , `project.md` , `rules.md`) are intentionally short. Deep content (`architecture.md` , `style.md` , skill playbooks, ADRs) lives in on-demand files the agent reads only when relevant.

The practical effect: a turn that asks "add a button" doesn't pay the cost of loading 1,500 lines of architecture docs. A turn that touches the data layer does load `architecture.md` , and that load is justified.

3. Onboarding for humans, not just agents

The same files that orient an AI assistant orient a new engineer. `.ai/project.md` , `.ai/architecture.md` , and the ADRs in `.ai/decisions/` are exactly what you'd hand a new hire on day one. The AI's source of truth and the team's onboarding doc are the same doc.

4. Future-proof against new agents

When the next coding assistant arrives, you add one shim file pointing into `.ai/` . The shim is typically 20–30 lines. You don't rewrite anything.

5. Skills capture institutional knowledge

The `.ai/skills/` folder turns repeated instructions ("when you add a feature, also update the OpenAPI spec, and run the codegen") into reusable playbooks. The second time you find yourself giving the same instruction, write a skill.

6. ADRs prevent re-litigation

`.ai/decisions/` records *why* past calls were made. AI assistants — like new engineers — often try to "fix" things that look odd but were deliberate. An ADR pre-empts that argument.

7. Works with what you already use

No new tools, no SaaS dependency, no CLI to install. It's a folder of markdown and YAML in your repo.

5. Project layout

Here is the complete file tree, annotated.

```
.
├── README.md
├── AGENTS.md
├── .ai/
│   ├── README.md
│   ├── project.md
│   ├── rules.md
│   ├── architecture.md
│   ├── style.md
│   ├── glossary.md
│   ├── skills/
│   │   ├── README.md
│   │   ├── add-feature.md
│   │   ├── fix-bug.md
│   │   ├── refactor.md
│   │   └── write-tests.md
│   └── decisions/
│       ├── README.md
│       └── TEMPLATE.md
├── CLAUDE.md
├── CONVENTIONS.md
├── .aider.conf.yml
├── .cursorrules
├── .cursor/rules/project.mdc
├── .github/copilot-instructions.md
├── .windsurfrules
├── .clinerules
└── .gitignore
```

← human-facing readme
← universal agent entry-point
(always loaded by most agents)

← SOURCE OF TRUTH (you edit these)
how the .ai/ folder works
what the project is, stack, commands
hard do / don't rules
system architecture
code style guide
domain terms
reusable workflows

architecture decision records

← Claude Code shim
← Aider shim
← Aider config
← Cursor (legacy) shim
← Cursor (modern) shim
← GitHub Copilot shim
← Windsurf shim
← Cline shim
← sensible defaults for secrets, etc.

The two folders to know

Folder	What lives here	How often you edit it
<code>.ai/</code>	Everything substantive: project description, rules, style, skills, decisions.	Often — this is your working surface.
Root-level shims	Thin pointer files for each agent.	Almost never. Only when adding a new agent.

6. How it works under the hood

The auto-load chain

Each AI coding tool has a designated file (or files) it loads into context automatically when you open the project. Here's what each one in this boilerplate does:

Agent	Auto-loads	Mechanism
Claude Code	CLAUDE.md	Uses <code>@path</code> imports to pull in <code>AGENTS.md</code> + key <code>.ai/</code> files.
Cursor (modern)	<code>.cursor/rules/project.mdc</code>	Frontmatter <code>alwaysApply: true</code> ; uses <code>@file</code> references.
Cursor (legacy)	<code>.cursorrules</code>	Plain text; instructs the assistant to read <code>AGENTS.md</code> + <code>.ai/</code> .
GitHub Copilot	<code>.github/copilot-instructions.md</code>	Copilot doesn't natively import, so this shim <i>summarizes</i> the top rules and links the rest.
Windsurf	<code>.windsurfrules</code>	Plain text; points at <code>AGENTS.md</code> + <code>.ai/</code> .
Cline	<code>.clinerules</code>	Plain text; points at <code>AGENTS.md</code> + <code>.ai/</code> .
Aider	<code>CONVENTIONS.md</code> via <code>.aider.conf.yml</code> 's <code>read:</code> directive	Aider's config explicitly preloads <code>AGENTS.md</code> , <code>.ai/project.md</code> , <code>.ai/rules.md</code> .
Codex / Jules / OpenAI Agents	<code>AGENTS.md</code>	Native — these agents read <code>AGENTS.md</code> as the standard.

Why the shims exist

You might wonder: if `AGENTS.md` is becoming a standard, why have shims at all? Two reasons:

1. **Not all agents auto-load `AGENTS.md` yet.** Cursor, Copilot, Windsurf, Cline, and Claude Code each look for their own filename first. The shim ensures every agent picks up the same context.

2. **Some agents support imports, others don't.** Claude Code can `@import` other files; Copilot pastes its instructions file in as-is. The shim is where we encode that per-agent quirk — without leaking it into `.ai/`.

The load-frequency split

Every file in this boilerplate falls into one of two tiers:

Tier	Files	Rule
Always loaded	<code>AGENTS.md</code> , <code>.ai/project.md</code> , <code>.ai/rules.md</code>	Keep short . Aim for under ~150 lines combined. Every line costs tokens on every turn.
On demand	<code>.ai/architecture.md</code> , <code>.ai/style.md</code> , <code>.ai/glossary.md</code> , <code>.ai/skills/*</code> , <code>.ai/decisions/*</code>	Length is fine. These load only when the agent decides they're relevant to the current task.

Design principle

Always-loaded files should answer "what exists and where to look." On-demand files contain the depth. If you find yourself adding a 50-line section to an always-loaded file, ask whether it belongs in `style.md` or a skill instead.

7. Quick start (5 minutes)

1. Clone or copy into your project

If you're starting a new repo:

```
git clone https://github.com/arjun8822/ai_wireframe.git my-project
cd my-project
rm -rf .git
git init
```

If you're adding to an existing repo, copy the files in (avoiding any conflicts with your existing `.github/`):

```
cp -R ai_wireframe/.ai ./
cp ai_wireframe/AGENTS.md ai_wireframe/CLAUDE.md ai_wireframe/CONVENTIONS.md ./
cp ai_wireframe/.aider.conf.yml ai_wireframe/.cursorrules
ai_wireframe/.clinerules ai_wireframe/.windsurfrules ./
cp -R ai_wireframe/.cursor ./
mkdir -p .github && cp ai_wireframe/.github/copilot-instructions.md .github/
```

2. Fill in `.ai/project.md`

This is the most important file. Replace every `<!-- FILL: ... -->` placeholder with your project's actual details: stack, run/test/build commands, entry points.

Why first: wrong commands here mean every agent wastes its first turn running the wrong command. Right commands here mean every agent gets it right on the first try.

3. Tune `.ai/rules.md`

The defaults are a sensible starting point. Add any project-specific hard rules to the "Always" / "Never" / "Ask before" sections. Keep the file under ~50 lines of rules — if it's longer, push detail into `style.md` or a skill.

4. Skim the rest

Open `.ai/architecture.md`, `.ai/style.md`, `.ai/glossary.md`. Fill what's relevant, delete what isn't. None of these are mandatory — an empty file with just a comment "no project-specific rules yet" is valid.

5. Commit

```
git add .  
git commit -m "chore: add AI coding boilerplate"
```

That's it. Open the project in Claude Code, Cursor, or any of the supported assistants — they will pick up the rules automatically.

8. Filling in the templates

Placeholders

Every template uses HTML comments as placeholders, like this:

```
<!-- FILL: 2-4 sentences. What is the product? Who uses it? -->
```

Replace the comment with your content. The comment itself can be deleted, or kept as a reminder while drafting.

`.ai/project.md` — the essentials

Sections, in priority order:

- **What this project is** — two to four sentences. Skip marketing copy. The agent needs enough to make sensible suggestions, not a pitch.
- **Stack** — framework, language, database, package manager, with *versions*. Without versions, the agent will hallucinate API shapes from its training data.
- **Run / test / build commands** — literal commands. Copy-paste from your real workflow.
- **Entry points** — the files an agent should read first to understand the shape of the codebase.

`.ai/rules.md` — the guardrails

Three sections: "Always", "Never", "Ask before". One line per rule. Add a short *why* only when it's not obvious.

Tip

When you correct the AI on the same thing twice, add it as a rule. The third time is too late.

`.ai/architecture.md` — the map

Don't restate the schema or the file tree. Focus on:

- The one-paragraph overview — "how does this system fit together?"
- What each component *is not* responsible for. This is the single most useful kind of content in an architecture doc.

- Choices that look naive but were deliberate — link to ADRs.

`.ai/style.md` — what the linter doesn't catch

If your formatter handles it (Prettier, ruff, Biome), don't restate it here. This file is for the things *not* enforced by tooling: error-handling style, comment policy, "forbidden patterns" that look fine to a generic AI but are wrong for your project.

`.ai/glossary.md` — the domain language

10–40 entries. Each is a bold term and a one-sentence definition. Include any term that means something specific in your project that it doesn't mean elsewhere — that's where AI assistants make their best-disguised mistakes.

9. Working with skills

What is a skill?

A skill is a short, scenario-specific playbook for a recurring task. Skills earn their place by being *repeated*. If you've given the same instructions to the AI more than twice, capture them as a skill.

The four bundled skills

Skill	When the agent loads it	What's inside
<code>add-feature.md</code>	Starting a feature	Confirm scope → find precedent → implement layer by layer → checklist before declaring done.
<code>fix-bug.md</code>	Investigating a bug	Reproduce first → find root cause not symptom → write failing test → minimal fix.
<code>refactor.md</code>	Doing a refactor	Confirm goal → check coverage → one transformation per step → don't bundle behavior changes.
<code>write-tests.md</code>	Adding tests	Test behavior not implementation → pyramid guidance → what to avoid (snapshot abuse, mocking your own code).

How agents pick up skills

`AGENTS.md` contains a routing table ("If you're doing X, read this skill first"). The agent consults it when starting a task and loads the matching skill into context.

Adding your own skill

1. Copy `.ai/skills/add-feature.md` as a starting structure.
2. Name the file after the task in imperative kebab-case: `add-migration.md`, `update-api-client.md`, `add-feature-flag.md`.
3. Keep it under ~100 lines. If it's longer, the task is probably two skills.
4. Add a row to the table in `.ai/skills/README.md`.
5. If the skill is frequently needed, also link it from the "When to load more context" table in `AGENTS.md`.

What makes a good skill

Self-contained: an agent loading *only* this skill should have enough to do the task. Concrete: orders of operation matter (write the failing test *before* the fix), so spell them out. Honest about exit criteria: every skill ends with "how do you know you're done."

10. Architecture decision records

Why ADRs exist in this boilerplate

AI assistants are pattern-matchers. When they encounter code that *looks* wrong by generic standards — an unused-looking abstraction, a "missing" layer, a duplicated helper — their instinct is to fix it. Sometimes they're right. Sometimes the oddity was deliberate and the fix re-introduces a bug the team solved six months ago.

An ADR (Architecture Decision Record) is a short, dated document explaining *why* a non-obvious choice was made. When the agent (or a new engineer) is about to "fix" something, an ADR is what stops them and tells them the real story.

When to write one

Write an ADR when:

- The choice is hard to reverse (database, framework, deployment target).
- A reasonable engineer would make the opposite call, and you need to document why you didn't.
- Future you / a new hire / an AI would otherwise undo the decision thinking it was an oversight.

Don't write one for:

- Routine library picks with cheap reversal cost.
- Decisions already encoded in the type system or in comments.
- Things already covered in `project.md` ("we use Postgres").

The template

`.ai/decisions/TEMPLATE.md` has five sections: *Status*, *Context*, *Decision*, *Consequences*, *Alternatives considered*. Keep each ADR short — one page is plenty.

The cardinal rule

Once an ADR is **accepted**, it's immutable. If a future decision overturns it, write a new ADR that supersedes the old one, and update the old one's status header to point at the new one. This way the historical record stays intact — future readers can see the path of reasoning, not just the latest answer.

11. Token optimization in depth

The cost model

Every AI coding assistant injects your "rules" or "instructions" file into the model's context on *every turn*. A 2,000-line CLAUDE.md adds ~2,000 lines of prompt to every message you send. That's:

- **Money:** token cost per turn, multiplied by every turn for the life of the project.
- **Latency:** more tokens = slower first response.
- **Quality:** the model's attention budget is finite. A long always-loaded preamble dilutes focus on the actual question.

The two-tier system

This boilerplate's central design choice is splitting content by load frequency:

Tier	Files	Target size
Always loaded	AGENTS.md + .ai/project.md + .ai/rules.md	Combined < 300 lines
On demand	architecture.md , style.md , glossary.md , all skills, all ADRs	No limit. Be as thorough as you want.

The "where to look" pattern

Instead of putting architecture detail in the always-loaded files, the always-loaded files contain a *routing table*: "if you're touching the data layer, read `architecture.md` ." The agent does the lookup when it matters, not preemptively.

What to keep small

- `AGENTS.md` : project summary in one paragraph, top 5 rules, the routing table. That's it.
- `.ai/project.md` : stack, commands, entry points. No essay.
- `.ai/rules.md` : bullet points only. Reasoning belongs in linked docs.

What to push to on-demand

- Anything narrative or explanatory.

- Anything specific to a task type (testing → `write-tests.md` , performance → consider a perf skill).
- Historical context (→ ADRs).

Anti-pattern to avoid

Don't dump your entire style guide, every architectural diagram, and every ADR into `CLAUDE.md` "just so the agent has everything." It's the AI equivalent of reading the entire employee handbook before every Slack message.

12. Adding a new agent

When a new AI coding assistant shows up, here's the recipe to wire it in — usually under five minutes.

Step 1: find the agent's config location

Check the agent's docs for its "rules" or "instructions" file. Examples to date:

- Claude Code → `CLAUDE.md`
- Cursor → `.cursor/rules/*.mdc`
- Copilot → `.github/copilot-instructions.md`
- Windsurf → `.windsurfrules`
- Cline → `.clinerules`
- Aider → `CONVENTIONS.md` (referenced from `.aider.conf.yml`)

Step 2: create a shim

Use one of the existing shims as a template. The pattern is:

1. A one-sentence note that the source of truth is in `AGENTS.md` and `.ai/`.
2. A list of always-read files.
3. A list of on-demand files with when-to-read guidance.
4. A short "behavior" section restating the key rules.

Step 3: handle the agent's quirks

- **If the agent supports file imports** (Claude's `@path`, Cursor's `@file`), use them in the shim.
- **If the agent doesn't import** (Copilot), summarize the top rules *in* the shim, then link to the full versions.
- **If the agent has a config file** (Aider's `.aider.conf.yml`), use it to preload the always-loaded files.

Step 4: update the README

Add a row to the "Supported agents" table in the root `README.md` so future users know it's wired up.

Migrating an existing project's rules

If you already have a `CLAUDE.md` or `.cursorrules` with content:

1. Move the substantive content into the appropriate `.ai/` file (project description → `project.md`, rules → `rules.md`, etc.).
2. Replace the original file with this boilerplate's thin shim.
3. Delete duplicated content. The shim points to the canonical source.

13. Best practices

Editing

- **Edit in `.ai/`, never in the shims.** The shims are wired correctly — touching them risks breaking the auto-load chain. Substantive change goes in `.ai/`.
- **Cross-reference with relative links.** Use `[architecture.md](architecture.md)`, not "see the architecture doc." Links are navigable; prose isn't.
- **Date your decisions.** Files under `.ai/decisions/` include a date. The dated record is what makes ADRs useful months later.

Sizing

- **Always-loaded files: under 150 lines.** Combined under 300 lines. If you exceed this, push detail to an on-demand file.
- **Skills: under 100 lines.** If longer, the task is probably two skills.
- **ADRs: one page.** If you need more, the decision wasn't crisp enough.

Content

- **Rules are bullet points, not essays.** Reasoning goes in linked docs.
- **Say what *not* to do.** Negative rules (the "Never" list, "Not responsible for" in architecture, "Forbidden patterns" in style) are the highest-signal content because they're the things an AI will get wrong without them.
- **Capture corrections.** When you correct the AI on the same thing twice, write it down as a rule or skill. The third time is wasted breath.

Operational

- **Treat the boilerplate as living documentation.** A rule that hasn't been touched in 18 months is suspect — either it's still true (good) or the code has drifted (bad). Review periodically.
- **Don't commit secrets.** The bundled `.gitignore` covers the common cases (`.env`, `*.pem`, `credentials.json`). Don't disable these without thinking about it.
- **When a rule conflicts with the code, trust the code.** Update the rule. The running code is authoritative; the docs are a description.

14. FAQ & troubleshooting

Won't 8 config files clutter my repo?

Yes, slightly — though most are dotfiles that hide by default. The trade-off is real: a small amount of root clutter in exchange for not maintaining the same rules in 8 places. If your team only uses one assistant, you can safely delete the shims for the others. The `.ai/` folder remains useful on its own.

What if two agents have conflicting opinions?

They shouldn't, because they're reading the same source. If one agent ignores a rule that another respects, it's not a content problem — it's a model behavior problem. File it as feedback for the tool vendor; the shared source still helps you reproduce the issue.

My Copilot shim is summarizing the rules. Doesn't that duplicate content?

A small amount, yes — the top 5 hard rules appear both in `.ai/rules.md` and (paraphrased) in `.github/copilot-instructions.md`, because Copilot can't import other files. This is a deliberate trade-off: a small summarized duplication beats Copilot operating without any rules at all. Keep the summary short (5–10 lines) and update both when a top rule changes.

What about `AGENTS.md` for tools that don't auto-load it?

The shim for each non-`AGENTS.md` tool explicitly tells it to read `AGENTS.md` first. So even tools that don't natively support the standard still get the same entry point.

How do I know it's working?

Easiest check: open your project in the assistant and ask "what is this project?" The answer should reflect what you put in `.ai/project.md`. If the answer is generic, the assistant isn't loading the files — check that the shim file exists at the path the agent expects.

Can I use this in a monorepo?

Yes. Put the boilerplate at the repo root. For per-package overrides, you can add a smaller `CLAUDE.md` (or equivalent) inside a package directory — most agents combine multiple rules files when both exist.

What if I want a rule to apply to only some files?

The modern Cursor format (`.cursor/rules/*.mdc`) supports glob-based scoping via frontmatter. For other agents, the convention is to mention the scope inline in the rule ("In `src/server/`, never...").

How do I version-control the boilerplate itself?

The boilerplate *is* just files in your repo, so it's versioned with the rest of your project. If you want to pull upstream updates, you can occasionally diff against the source repo (github.com/arjun8822/ai_wireframe) and cherry-pick changes — usually to the bundled skills or the shims, since your `.ai/` content is yours.

Is this compatible with team-level rules (e.g. company-wide standards)?

Yes. Many agents support a layered rules system (user-level + project-level). The boilerplate sits at the project level. Org-wide rules live in the user-level location for each agent and combine with this project's rules at load time.

15. File reference

Source of truth (`.ai/`)

File	Purpose	Edit frequency
<code>.ai/README.md</code>	How the <code>.ai/</code> folder works.	Rare.
<code>.ai/project.md</code>	What the project is, stack, commands. Always loaded.	When stack/commands change.
<code>.ai/rules.md</code>	Hard do/don't rules. Always loaded.	When you correct the AI twice on the same thing.
<code>.ai/architecture.md</code>	System design. Loaded on demand.	When the architecture changes.
<code>.ai/style.md</code>	Project-specific style not enforced by formatters. On demand.	When style conventions shift.
<code>.ai/glossary.md</code>	Domain terms. On demand.	When new terms appear.
<code>.ai/skills/*.md</code>	Per-task playbooks. On demand.	When you find yourself repeating instructions.
<code>.ai/decisions/*.md</code>	Architecture decision records. On demand.	Append-only. New ADRs supersede old ones.

Universal entry-point

File	Purpose
<code>AGENTS.md</code>	Short, always-loaded entry point. Top rules + routing table. Read by <code>AGENTS.md</code> -aware agents natively; referenced by all the shims.

Agent shims

File	Agent	Mechanism
<code>CLAUDE.md</code>	Claude Code	Uses <code>@path</code> imports.

<code>.cursorrules</code>	Cursor (legacy)	Plain text instruction file.
<code>.cursor/rules/project.mdc</code>	Cursor (modern)	Frontmatter <code>alwaysApply: true</code> + <code>@file</code> refs.
<code>.github/copilot-instructions.md</code>	GitHub Copilot	Plain markdown; top rules summarized inline.
<code>.windsurfrules</code>	Windsurf	Plain text instruction file.
<code>.clinerules</code>	Cline	Plain text instruction file.
<code>CONVENTIONS.md</code>	Aider	Conventions file referenced by <code>.aider.conf.yml</code> .
<code>.aider.conf.yml</code>	Aider	YAML config; preloads <code>CONVENTIONS.md</code> , <code>AGENTS.md</code> , <code>.ai/project.md</code> , <code>.ai/rules.md</code> .

Repo housekeeping

File	Purpose
<code>README.md</code>	Human-facing readme for the boilerplate itself.
<code>.gitignore</code>	Sensible defaults: ignores secrets, IDE state, build output, AI tool caches.
<code>docs/guide.pdf</code>	This document.